

UNIT 2: Object-Oriented Programming Concepts

Q1. Define Constructor. List types of Constructors. Develop a java code to explain Default constructor. [07 Marks]

(Summer-2026)

Answer:

Definition of Constructor:

- Constructor એ class ની એક special method છે જેનો ઉપયોગ **Object** ને **initialize** કરવા માટે થાય છે.
- જ્યારે new operator નો ઉપયોગ કરીને class નો object બનાવવામાં આવે છે ત્યારે constructor આપમેળે (automatically) call થાય છે.
- **Rules to define Constructor:**
 - Constructor નું નામ અને Class નું નામ સરખું જ હોવું જોઈએ.
 - Constructor ને કોઈ પણ return type હોતી નથી (તે void પણ ધરાવી શકતો નથી).

Types of Constructors in Java:

1. **Default Constructor (No-argument constructor):** જેમાં એક પણ parameter હોતો નથી.
2. **Parameterized Constructor:** જેમાં arguments/parameters પાસ કરવામાં આવે છે.
3. **Copy Constructor:** જે constructor માં તે જ class નો object parameter તરીકે પાસ થાય છે.

Java Code for Default Constructor:

```
Java
class Student {
    int rollNo;
    String name;
```

```
// Default constructor (No parameters)
Student() {
    rollNo = 101;
    name = "Sujal";
    System.out.println("Default Constructor called..");
}

void display() {
    System.out.println("Roll No: " + rollNo);
    System.out.println("Name: " + name);
}
}

public class DefaultConsDemo {
    public static void main(String args[]) {
        // Creating object invokes Default Constructor automatically
        Student s1 = new Student();
        s1.display();
    }
}
```

Output:

Plaintext
Default Constructor called..
Roll No: 101
Name: Sujal

Q2. Define Constructor. List types of Constructors. Develop a java code to explain Parameterized constructor. [07 Marks]

(Winter-2024)

Answer:

Definition: Constructor એ class નો એક વિશિષ્ટ member છે જે object creation વખતે data members ને initialize કરવાનું કાર્ય કરે છે.

Types of Constructors:

1. Default Constructor
2. Parameterized Constructor
3. Copy Constructor

Parameterized Constructor:

- જે constructor ની અંદર parameters/arguments પાસ કરવામાં આવે છે તેને **Parameterized Constructor** કહેવાય છે.
- તેનો ઉપયોગ જુદા જુદા objects ને જુદી જુદી શરૂઆતની values (initial values) assign કરવા માટે થાય છે.

Java Code for Parameterized Constructor:

Java

```
class Employee {
    int empId;
    String empName;

    // Parameterized Constructor
    Employee(int id, String name) {
        empId = id;
        empName = name;
    }

    void display() {
        System.out.println("Employee ID: " + empId + ", Name: " +
empName);
    }
}

public class ParamConsDemo {
    public static void main(String args[]) {
        // Passing arguments during object creation
        Employee e1 = new Employee(1, "Rahul");
        Employee e2 = new Employee(2, "Amit");
    }
}
```

```
        e1.display();
        e2.display();
    }
}
```

Output:

Plaintext

Employee ID: 1, Name: Rahul

Employee ID: 2, Name: Amit

Q3. Define access modifiers. List the four types used in Java. [03 Marks]*(Winter-2025)***Answer:**

Definition: Access Modifiers (અથવા Visibility Controls) એ Java ના વિશિષ્ટ keywords છે જે classes, methods, constructors અને variables નું **Access Level (Visibility/Scope)** નક્કી કરે છે કે તેને ક્યાંથી access કરી શકાય.

The Four Types of Access Modifiers:

1. **private:** Members ફક્ત તે જ class ની અંદર access થાય છે.
2. **default (Package-private / No keyword):** Members ફક્ત તે જ package ની અંદર access થઈ શકે છે.
3. **protected:** Members તે જ package ની અંદર તથા બીજા package માં રહેલા derived (subclass) માં access થાય છે.
4. **public:** Members સમગ્ર Java પ્રોગ્રામમાં (કોઈપણ class કે package માંથી) access થઈ શકે છે.

Q4. Define: Constructor. List out types of it. Explain Parameterized and copy constructor with suitable example. [07 Marks]*(Winter-2025)*

Answer:

Definition: Constructor એ class ની એવી method છે જેનું નામ class ના નામ જેવું જ હોય છે અને તેનો ઉપયોગ object instance ને initialize કરવા માટે થાય છે.

Types of Constructors:

1. Default Constructor
2. Parameterized Constructor
3. Copy Constructor

Explanation of Types:

- **Parameterized Constructor:** જુદા જુદા objects માટે dynamic initial values સેટ કરવા માટે parameters સ્વીકારે છે.
- **Copy Constructor:** તે જ class ના existing object ને parameter તરીકે સ્વીકારે છે અને તેની duplicate copy બનાવે છે જે original object થી સ્વતંત્ર memory location ધરાવે છે.

Complete Program:

Java

```
class Box {  
    double width, height, depth;  
  
    // Parameterized Constructor  
    Box(double w, double h, double d) {  
        width = w;  
        height = h;  
        depth = d;  
    }  
  
    // Copy Constructor (Passing Box object as argument)  
    Box(Box ob) {  
        width = ob.width;  
        height = ob.height;  
        depth = ob.depth;  
    }  
}
```

```
double volume() {  
    return width * height * depth;  
}  
}  
  
public class ConstypesDemo {  
    public static void main(String args[]) {  
        // Using Parameterized Constructor  
        Box b1 = new Box(10, 20, 15);  
        System.out.println("Volume of b1: " + b1.volume());  
  
        // Using Copy Constructor (Pass b1 into new object)  
        Box b2 = new Box(b1);  
        System.out.println("Volume of b2 (Clone): " + b2.volume());  
    }  
}
```

Output:

Plaintext

Volume of b1: 3000.0

Volume of b2 (Clone): 3000.0

Q5. Differentiate between Procedure-Oriented Programming (POP) and Object-Oriented Programming (OOP). [03 Marks]*(Winter-2024, Winter-2025)***Answer:**

Feature	Procedure-Oriented (POP) PDF	Object-Oriented (OOP) PDF
Focus	Functions અને Procedures પર વધુ ધ્યાન અપાય છે.	Data પર સૌથી વધુ ધ્યાન આપવામાં આવે છે.
Approach	Top-Down ડિઝાઇન ધરાવે છે.	Bottom-Up ડિઝાઇન ધરાવે છે.

Division	પ્રોગ્રામ Functions માં વિભાજિત થાય છે.	પ્રોગ્રામ Classes અને Objects માં વહેંચાય છે.
Security	Global data હોવાથી data સુરક્ષિત રહેતો નથી.	Data Encapsulation અને Access Modifiers થી data hide રહે છે.
Concepts	Inheritance કે Polymorphism હોતા નથી.	Inheritance, Polymorphism, Abstraction હાજર હોય છે.
Examples	C, Fortran, Pascal	Java, C++, C#

Q6. Explain Garbage collection in java. [03 Marks]

(Summer-2024, Winter-2025)

Answer:

- Java માં new operator દ્વારા dynamically allocate થયેલા objects ની memory મુક્ત કરવાનું કાર્ય સ્વયંચાલિત છે, જેને **Garbage Collection (GC)** કહે છે.
- C++ માં વપરાતી delete operator ની જેમ Java માં manually memory release કરવાની જરૂર રહેતી નથી.
- **Working:** જ્યારે કોઈ object નો reference variable null થઈ જાય અથવા કોઈ reference બાકી ન રહે, ત્યારે JVM તે object ને unused ગણીને તેની memory recovered (free) કરી લે છે.
- Execution દરમિયાન Garbage Collector બેકગ્રાઉન્ડ daemon thread તરીકે રન થઈને memory leaks અટકાવે છે.

Q7. Explain concept of static word and design a Java program that uses static blocks, static variables, and static methods. [07 Marks]

(Winter-2025)

Answer:

Concept of static: static keyword નો ઉપયોગ મુખ્યત્વે **Memory Management** માટે થાય છે. જ્યારે કોઈ member static બને છે, ત્યારે તે ચોક્કસ object નો નહિ પરંતુ સમગ્ર **Class** નો member બને છે.

1. **Static Variable:** Class ની તમામ objects વચ્ચે single memory copy share થાય છે.
2. **Static Block:** Class memory માં load થતી વખતે main method અને constructor પહેલાં એક જ વાર execute થાય છે.
3. **Static Method:** Object બનાવ્યા વિના સીધી `ClassName.methodName()` થી call કરી શકાય છે. તે ફક્ત static data access કરી શકે છે.

Java Program:

Java

```
class StaticDemo {
    // 1. Static Variable
    static int count = 0;

    // 2. Static Block
    static {
        System.out.println("Static block initialized...");
    }

    // Constructor
    StaticDemo() {
        count++;
        System.out.println("Current count: " + count);
    }

    // 3. Static Method
    static void displayMessage() {
        System.out.println("Static method called without object.");
    }
}

public class MainStaticTest {
    public static void main(String args[]) {
```

```

    // Calling static method directly using class name
    StaticDemo.displayMessage();

    // Creating objects to demonstrate shared static variable
    StaticDemo s1 = new StaticDemo();
    StaticDemo s2 = new StaticDemo();
}
}

```

Output:

```

Plaintext
Static block initialized...
Static method called without object.
Current count: 1
Current count: 2

```

Q8. Explain different access controls in Java. [04 Marks]

(Summer-2024, Winter-2025)

Answer:

Java માં member ની visibility નિયંત્રિત કરવા માટે નીચેના 4 access controls છે:

Access Modifier	Within Class	Within Package Subclass	Within Package Non-Subclass	Outside Package Subclass	Outside Package Non-Subclass
private	Yes	No	No	No	No
default	Yes	Yes	Yes	No	No
protected	Yes	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes	Yes

- **1. private:** ડેટાને સંપૂર્ણપણે hide કરે છે. ફક્ત declared class માં જ વપરાય.

- **2. default:** કોઈ modifier ન લખતી default ગણાય. ફક્ત same package માં જ visible થાય છે.
- **3. protected:** Same package માં તથા બીજા package ની inheritance પામેલા subclass માં access થઈ શકે.
- **4. public:** તમામ classes અને packages માટે મુક્તપણે accessible છે.

Q9. Explain final keyword with example. [04 Marks]

(Summer-2024, Winter-2024, Winter-2025, Summer-2026)

Answer:

final keyword restrictions લાદવા માટે વપરાય છે:

1. **Final Variable:** Variable ની કિંમત constant બને છે, તેને બદલી શકાતી નથી.
2. **Final Method:** Method ને subclass માં override કરી શકાતી નથી.
3. **Final Class:** Class ને inherit (extend) કરી શકાતો નથી.

Example Program:

Java

```
class FinalDemo {
    final int MAX_LIMIT = 100; // Final variable

    void changeValue() {
        // MAX_LIMIT = 200; // Compile-time Error: cannot assign a
        value to final variable
        System.out.println("Max Limit is: " + MAX_LIMIT);
    }

    public static void main(String args[]) {
        FinalDemo obj = new FinalDemo();
        obj.changeValue();
    }
}
```

Output:

Plaintext

Max Limit is: 100

Q10. Explain method overloading in JAVA with example. [04 Marks]

(Winter-2025)

Answer:

- **Concept:** જ્યારે એક જ class ની અંદર એક જ નામના એકાંશિક methods હોય પરંતુ તેમના **Parameters** ની સંખ્યા, **Data Type** અથવા **Sequence** અલગ હોય, તેને **Method Overloading** કહે છે.
- તે **Compile-time Polymorphism (Static Binding)** તરીકે ઓળખાય છે.
- ફક્ત return type બદલવાથી overloading થતું નથી.

Example Program:

Java

```
class MathOperations {
    // Method 1: Two integer parameters
    void sum(int a, int b) {
        System.out.println("Sum of 2 ints: " + (a + b));
    }

    // Method 2: Three integer parameters
    void sum(int a, int b, int c) {
        System.out.println("Sum of 3 ints: " + (a + b + c));
    }

    // Method 3: Two double parameters
    void sum(double a, double b) {
        System.out.println("Sum of 2 doubles: " + (a + b));
    }
}

public class OverloadDemo {
    public static void main(String args[]) {
        MathOperations m = new MathOperations();
    }
}
```

```
        m.sum(10, 20);           // Calls method 1
        m.sum(10, 20, 30);       // Calls method 2
        m.sum(10.5, 20.5);       // Calls method 3
    }
}
```

Output:

Plaintext

Sum of 2 ints: 30

Sum of 3 ints: 60

Sum of 2 doubles: 31.0

Q11. Explain static in java with example. [04 Marks]

(Summer-2026)

Answer:

Java માં static keyword નો ઉપયોગ class-level variables અને methods બનાવવા માટે થાય છે જે બધા objects વચ્ચે shared હોય છે.

Key Points:

1. Static variables ને class load થાય ત્યારે એક જ વાર memory મળે છે.
2. Static methods object બનાવ્યા વિના `ClassName.method()` થી call થાય છે.
3. Java ની `main()` method પણ static હોય છે જેથી JVM તેને object બનાવ્યા વગર સીધી call કરી શકે.

Example Program:

Java

```
class Student {
    int id;
    static String college = "GTU Polytechnic"; // Shared static
    variable

    Student(int i) {
```

```
        id = i;
    }

    void display() {
        System.out.println("ID: " + id + ", College: " + college);
    }
}

public class StaticSimpleDemo {
    public static void main(String args[]) {
        Student s1 = new Student(101);
        Student s2 = new Student(102);

        s1.display();
        s2.display();
    }
}
```

Output:

Plaintext

ID: 101, College: GTU Polytechnic

ID: 102, College: GTU Polytechnic

Q12. Explain static keyword with example. [04 Marks]

(Summer-2024, Winter-2024)

Answer:

static keyword નો ઉપયોગ variable, method કે block સાથે કરી શકાય છે:

Java

```
class StaticCounter {
    static int count = 0; // Gets memory only once

    StaticCounter() {
        count++;
        System.out.println("Count: " + count);
    }
}
```

```
}

static void showInfo() {
    System.out.println("Static method executed.");
}

}

public class StaticTest {
    public static void main(String args[]) {
        StaticCounter.showInfo(); // Direct class call
        StaticCounter c1 = new StaticCounter();
        StaticCounter c2 = new StaticCounter();
    }
}
```

Output:

Plaintext

Static method executed.

Count: 1

Count: 2

Q13. Explain the role of the final keyword in variables and methods. [03 Marks]

(Winter-2025)

Answer:**1. Role in Variables:**

- જ્યારે variable ને final declare કરવામાં આવે છે, ત્યારે તે **constant** બની જાય છે.
- તેની value સમગ્ર execution દરમિયાન એક જ વાર initialize થાય છે અને પછી modify કરી શકાતી નથી.
- Example: final double PI = 3.14159;

2. Role in Methods:

- જ્યારે કોઈ method ને final તરીકે declare કરવામાં આવે છે, ત્યારે તેને derived subclass માં ઓવરરાઇડ (**override**) કરી શકાતી નથી.

b. આ ચોક્કસ logic ની security જાળવવા માટે વપરાય છે.

Q14. Explain this in Java with example. [03 Marks]

(Summer-2026)

Answer:

- this એ એક reference variable છે જે method અથવા constructor માં **current object** ને refer કરે છે.
- જ્યારે instance variable અને local variable ના નામ એક જ હોય ત્યારે instance variable ને refer કરવા માટે this વપરાય છે.

Example:

Java

```
class Demo {  
    int v = 5; // Instance variable  
  
    void method(int v) { // Local variable  
        System.out.println("Instance variable: " + this.v);  
        System.out.println("Local variable: " + v);  
    }  
  
    public static void main(String args[]) {  
        Demo d = new Demo();  
        d.method(20);  
    }  
}
```

Output:

Plaintext

Instance variable: 5

Local variable: 20

Q15. Explain this keyword with suitable example. [03 Marks]*(Summer-2024)***Answer:**

this keyword નો ઉપયોગ constructor માં instance variable initialize કરવા માટે બહોળા પ્રમાણમાં થાય છે:

Java

```
class Rectangle {
    int length, width;

    Rectangle(int length, int width) {
        // Differentiating instance variables from local parameters
        this.length = length;
        this.width = width;
    }

    void area() {
        System.out.println("Area: " + (this.length * this.width));
    }

    public static void main(String args[]) {
        Rectangle r = new Rectangle(5, 4);
        r.area();
    }
}
```

Output:

Plaintext

Area: 20

Q16. Give four Difference between Procedure Oriented Programming and Object-Oriented Programming. [04 Marks]*(Summer-2024, Summer-2025)*

Answer:

No.	Procedure-Oriented Programming (POP) PDF	Object-Oriented Programming (OOP) PDF
1	Functions અને procedures ને મુખ્ય મહત્વ આપે છે.	Data ને સૌથી વધુ પ્રાધાન્ય આપે છે.
2	Program design Top-Down હોય છે.	Program design Bottom-Up હોય છે.
3	Access Specifiers હોતા નથી.	public, private, protected access specifiers હોય છે.
4	Data અને function ઉમેરવા મુશ્કેલ છે.	Data અને function સરળતાથી ઉમેરી શકાય છે.

Q17. Give the basic syntax of the following terms with an example: (1) To create a Class, (2) To create an Object, (3) To define a Method, (4) To declare a Variable. [04 Marks]

(Summer-2025)

Answer:

1. To create a Class:

a. **Syntax:** class ClassName { /* members */ }

b. **Example:** class Student { int id; }

2. To create an Object:

a. **Syntax:** ClassName objectName = new ClassName();

b. **Example:** Student s1 = new Student();

3. To define a Method:

a. **Syntax:** return_type methodName(parameter_list) { /* body */ }

b. **Example:**

Java

```
void display() {
    System.out.println("Hello");
}
```

```
}
```

4. To declare a Variable:

- a. **Syntax:** `data_type variable_name [= value];`
- b. **Example:** `int marks = 85;`

Q18. List the basic OOP concepts in Java and explain any one. [03 Marks]

(Winter-2024)

Answer:

Basic OOP Concepts:

1. Class
2. Object
3. Encapsulation
4. Abstraction
5. Inheritance
6. Polymorphism

Explanation of Encapsulation (ઇનકેપ્સ્યુલેશન):

- ડેટા (variables) અને તેના પર કામ કરતી methods ને એક સિંગલ યુનિટ (class) માં wrap up (બાઈન્ડ) કરવાની પ્રક્રિયાને **Encapsulation** કહે છે.
- તે ડેટાને બહારની દુનિયાથી protect કરે છે (Data Hiding). Variables ને private બનાવી getter/setter methods દ્વારા એક્સેસ આપીને Encapsulation પ્રાપ્ત થાય છે.

Q19. List the differences between Procedure-Oriented Programming (POP) and Object-Oriented Programming (OOP). [03 Marks]

(Winter-2024)

Answer:

- POP મોટા પ્રોગ્રામ્સને **Functions** માં વિભાજિત કરે છે, જ્યારે OOP પ્રોગ્રામ્સને **Classes** અને **Objects** માં વહેંચે છે.

- POP માં data global હોવાથી function થી function freely flow થાય છે, જ્યારે OOP માં access specifiers દ્વારા data access control થાય છે.
- POP Top-Down design વાપરે છે, જ્યારે OOP Bottom-Up design વાપરે છે.

Q20. What is constructor? Explain parameterized constructor with example. [07 Marks]

(Summer-2024)

Answer:

Constructor: Constructor એ class ની એક special method છે જે object બનતી વખતે automatically call થાય છે અને memory માં variable initialize કરે છે.

Parameterized Constructor: જે constructor માં arguments પાસ કરી શકાય તેને Parameterized Constructor કહે છે. તે દરેક object ને અલગ attributes assign કરવા માટે વપરાય છે.

Program:

Java

```
class Box {  
    double width, height, depth;  
  
    // Parameterized constructor  
    Box(double w, double h, double d) {  
        width = w;  
        height = h;  
        depth = d;  
    }  
  
    double volume() {  
        return width * height * depth;  
    }  
}
```

```
public class BoxTest {
```

```
    public static void main(String args[]) {  
        Box b1 = new Box(10, 20, 15);  
        double vol = b1.volume();  
        System.out.println("Volume of Box is: " + vol);  
    }  
}
```

Output:

Plaintext

Volume of Box is: 3000.0

**Q21. Write a program in Java that shows the use of Default Constructor.
[03 Marks]**

(Summer-2025)

Answer:

Java

```
class College {  
    String name;  
  
    // Default constructor  
    College() {  
        name = "Government Polytechnic";  
        System.out.println("Default constructor called.");  
    }  
  
    void show() {  
        System.out.println("College Name: " + name);  
    }  
}  
  
public class DefaultDemo {  
    public static void main(String args[]) {  
        College c = new College(); // Calls default constructor  
        c.show();  
    }  
}
```

```
}
```

Output:

Plaintext

Default constructor called.

College Name: Government Polytechnic

Q22. Write a program in Java that shows the use of parameterized Constructor. [03 Marks]*(Summer-2025)***Answer:**

Java

```
class Circle {
    double radius;

    // Parameterized constructor
    Circle(double r) {
        radius = r;
    }

    void calcArea() {
        System.out.println("Area of Circle: " + (3.14 * radius *
radius));
    }
}

public class ParamDemo {
    public static void main(String args[]) {
        Circle c1 = new Circle(7.0); // Passing argument
        c1.calcArea();
    }
}
```

Output:

Plaintext

Area of Circle: 153.86

Q23. Write a program in Java which has a class Student having two instance variables enrollmentNo and name. Create 3 objects of Student class in main method and display student's name. [07 Marks]

(Summer-2025)

Answer:

Java

```
class Student {
    // Two instance variables
    long enrollmentNo;
    String name;

    // Constructor to initialize variables
    Student(long eNo, String sName) {
        enrollmentNo = eNo;
        name = sName;
    }

    void display() {
        System.out.println("Enrollment No: " + enrollmentNo + " |
Student Name: " + name);
    }
}

public class StudentDisplayDemo {
    public static void main(String args[]) {
        // Creating 3 objects of Student class
        Student s1 = new Student(230001001L, "Sujal");
        Student s2 = new Student(230001002L, "Rahul");
        Student s3 = new Student(230001003L, "Amit");

        // Displaying names
        System.out.println("Student Details:");
        s1.display();
        s2.display();
    }
}
```

```
        s3.display();
    }
}
```

Output:

Plaintext

Student Details:

Enrollment No: 230001001 | Student Name: Sujal

Enrollment No: 230001002 | Student Name: Rahul

Enrollment No: 230001003 | Student Name: Amit

Q24. Write a short note on constructor overloading with an example program. [04 Marks]

(Winter-2025)

Answer:

- **Concept:** જ્યારે એક જ class ની અંદર એક કરતાં વધારે constructors અલગ-અલગ parameters (signature) સાથે define કરવામાં આવે તેને **Constructor Overloading** કહે છે.
- Method overloading ની જેમ જ કયો constructor call થશે તે passing arguments ની પ્રકાર અને સંખ્યા પર આધાર રાખે છે.
- Constructor ને કોઈ return type હોતી નથી.

Example Program:

Java

```
class Box {
    double width, height, depth;

    // Constructor 1: Default
    Box() {
        width = height = depth = 0;
    }
}
```

```
// Constructor 2: Cube
Box(double len) {
    width = height = depth = len;
}

// Constructor 3: Parameterized box
Box(double w, double h, double d) {
    width = w;
    height = h;
    depth = d;
}

double volume() {
    return width * height * depth;
}

}

public class ConsOverloadTest {
    public static void main(String args[]) {
        Box b1 = new Box();           // Calls Constructor 1
        Box b2 = new Box(5);          // Calls Constructor 2
        Box b3 = new Box(2, 3, 4);     // Calls Constructor 3

        System.out.println("b1 Volume: " + b1.volume());
        System.out.println("b2 Volume (Cube): " + b2.volume());
        System.out.println("b3 Volume: " + b3.volume());
    }
}
```

Output:

Plaintext

b1 Volume: 0.0

b2 Volume (Cube): 125.0

b3 Volume: 24.0

Q25. Write scope of java access modifier. Develop a java code to explain public modifier. [07 Marks]*(Winter-2024)***Answer:****Scope of Java Access Modifiers:**

- **private:** ફક્ત પોતાના class પૂરતો જ સીમિત.
- **default:** ફક્ત પોતાના package પૂરતો જ સીમિત.
- **protected:** Same package અને અન્ય package ની derived subclasses માં access થાય છે.
- **public:** કોઈપણ class, કોઈપણ package કે subpackage માં સર્વત્ર accessible છે.

Java Code explaining public modifier:

Java

```
// Class declared as public
public class PublicScopeDemo {
    // Public instance variable
    public String welcomeMessage = "Access Granted: This is Public
Data!";

    // Public method
    public void displayMessage() {
        System.out.println(welcomeMessage);
    }

    public static void main(String args[]) {
        PublicScopeDemo obj = new PublicScopeDemo();
        // Public method & variable can be accessed everywhere
        obj.displayMessage();
    }
}
```

Output:

Plaintext

Access Granted: This is Public Data!